

Final Report

**<2520-0027> < Extracting and Reconciling Industrial Operations Data with AI:
Automotive Manufacturing and Supply chain>**

Submitted by

Chen Jung-Lu	A0331628E
Jin Hongquan	A0329547R
Zhang Zhiyuan	A0328241L
Mo Yuyang	A0331388Y
Ye Wenrui	A0328745U

Supervised by

Prof Jussi Keppo
Department of Analytics & Operations
NUS Business School

Project Sponsor

Staple AI Pte Ltd

Submitted on

Jun 12, 2026

Abstract

This capstone project evaluates how Fibula, a stateful node-based automation workspace developed with Staple AI, can support industrial document reconciliation beyond basic AI extraction. In Smart Industry 4.0 environments, documents such as purchase orders, goods receipt notes, invoices, and BOM records are often manually cross-verified, creating administrative friction, delays, and financial leakage. Although modern IDP systems and LLM-based extraction tools can process complex documents, they often fail to support users when variances occur, leaving non-technical operators with low visibility and limited trust in the workflow. To address this gap, the team assessed Fibula through AP, BOM, and supplier non-conformance reconciliation use cases. An 8-week HCI evaluation sprint using task-based testing, think-aloud observation, and a no-help protocol identified three main adoption barriers: spatial desynchronization between chat and canvas, choice fatigue during setup, and operational opacity during exception handling. Based on these findings, the project proposes improvements such as clearer workflow synchronization, reduced option complexity, and natural-language recovery guidance. Overall, the project shows that effective reconciliation automation requires more than accurate extraction. It requires stateful workflow logic, visible system reasoning, and user-centered guidance that enables non-technical operators to understand, trust, and act on AI-generated reconciliation outcomes.

Chapter 1 Introduction

1.1. Operational Context: Document Reconciliation as Human Work

This chapter begins with the localized problem encountered in the Staple AI capstone environment. Staple AI operates in the intelligent document processing domain, where public descriptions emphasize document ingestion, extraction, reconciliation, ERP connectivity, multilingual

processing, and audit-oriented document automation [1, 2]. The capstone problem was not that artificial intelligence could not read a purchase order, invoice, delivery note, bill of materials, work order, or inspection record. The problem was that successful field extraction did not automatically produce an operator-usable reconciliation system. Once a variance appeared, the burden shifted from the model to the human user.

In this setting, document reconciliation is not a passive data-entry activity. It is a human decision process carried out under operational pressure. A warehouse clerk may need to determine why a goods receipt quantity does not match a supplier's invoice. An accounts payable accountant may need to decide whether a price variance is acceptable under policy. A production planner may need to explain why actual material issues exceed the bill-of-materials expectation. A quality coordinator may need to connect a non-conformance record to a supplier on chargeback. These users are not evaluating model accuracy as an abstract metric. They are trying to close an operational task without losing traceability or making an unauthorized financial decision.

The central user of friction appears when the system reports a mismatch without preserving a usable evidence trail. A generic exception such as 'mismatch, failed validation,' or 'processing error' forces the operator to reconstruct the task manually. The user must locate the original PDF. Identify the relevant tracking number, compare it against the purchase order or work order, check whether the issue is caused by partial shipment or unit conversion, and then decide whether the exception should be approved, rejected, corrected, or escalated. The cost of this work is cognitive before it is financial: the user must hold several document states in memory while navigating between fragmented files and system screens.

This cognitive load is the specific HCI problem that motivates Project Fibula. The operator is expected to take responsibility for the decision, but the interface often provides insufficient support for understanding the system's state. Human-AI interaction research stresses that AI systems should communicate what they can do, show uncertainty and limitations, support efficient correction, and maintain context when users need to intervene [4]. NIST similarly frames trustworthy AI in terms that include accountability, transparency, explainability, interpretability, validity, and reliability [3]. In document reconciliation, these principles translate into concrete interface requirements: the user must see which documents were matched, which fields of conflict, which policy was applied, what state the transaction is in, and what action is available.

The baseline issue is therefore not merely low on automation coverage. It is a mismatch between the structure of industrial reconciliation and the structure of many document automation tools. The business process is evidence-centered, exception-heavy, and human-in-the-loop. The technical workflow is often extraction-centered, linear, and brittle when variance occurs.

1.2. Problem Framing: Stateless Pipelines Versus Stateful Human-in-the-Loop Reconciliation

A fundamental limitation in contemporary document automation is the assumption that extraction accuracy correlates directly with system adoption. High extraction accuracy is necessary, but it is not sufficient for reconciliation to work. A system may correctly extract a PO number, supplier name, invoice total, quantity, unit price, currency, work order number, or inspection result while still failing to support the user who must decide what to do when two correct fields disagree. The adoption problem begins where extraction ends.

The architectural limitation can be stated precisely: conventional intelligent document processing workflows often behave as stateless data pipelines. In a stateless pipeline, the system ingests a file, extracts field, maps the fields into a payload, validates the payload against fixed rules, and either passes the payload downstream or emits an exception. Once a variance occurs, the process sequence is broken. The pipeline may record that a mismatch occurred, but it does not necessarily preserve an interactive runtime state in which the relevant documents, rule, variance, user decision, and resumed execution path remain bound together.

Industrial reconciliation is inherently stateful. A transaction remains unfinished until the system has resolved the relationship between multiple documents and any required human decision. If a PO states 10 units and the invoice states 12 units, the correct system behavior is not simply failing. The system must hold the transaction in a designated runtime state, retrieve or display the goods receipt evidence, evaluate the configured tolerance, request approval if the variance exceeds policy, record the user's decision, and then continue or reject the workflow accordingly. The document must live inside a reconciliation context rather than disappear into a failed pipeline.

This state requirement is also an HCI requirement. A user cannot confidently approve a variance if the interface shows only an error code. The interface must expose the reason for the pause, the data used by the AI, the documents supporting each field, the rule that produced the variance, and the consequences of each available action. Without this visibility, the user is asked to trust an automated system while simultaneously compensating for its failure. That design pattern produces low retention because it transfers accountability to the operator without transferring enough context.

The stateless-stateful distinction should therefore be treated as the central thesis of the introduction. Project Fibula is not justified because chat interfaces are fashionable or because node canvases are visually expressive. It is justified because reconciliation requires persistent runtime state. The system must support document intake, evidence binding, decision routing, approval holding, state resumption, and audit output in one coherent workflow. Natural language is valuable only if it lowers the barrier to constructing that stateful workflow while preserving inspect ability.

Figure 1 (appendix 2) represents the pre-Fibula baseline as a BPMN-style process. The operator can describe a new reconciliation of need, but the actual implementation of rule logic is mediated by technical staff. The IDP pipeline extracts fields and applies linear rules. When the exclusive gateway encounters a variance, the process emits a generic exception and returns the problem to a developer-mediated modification loop. The figure is intentionally centered on process ownership: the business user owns the reconciliation judgment, but the developer owns the automation configuration. This ownership split is the baseline bottleneck that the capstone addresses.

The failure is not that the pipeline cannot be repaired. The failure is that repair requires technical intervention whenever the business logic changes. In industrial environments, rule changes are routine: suppliers use new document layouts, plants introduce new material codes, partial shipments alter receipt patterns, finance teams adjust tolerances, and quality teams introduce new chargeback categories. If every rule of change requires a developer to select nodes, configure conditions, wire transitions, and redeploy the pipeline, the system remains in a technical service rather than a self-serve business application.

1.3. Project Scope: Staple AI Baseline and Multi-Source Reconciliation Use Cases

The relevant pre-Fibula baseline at Staple AI is developer-dependent workflow authoring for new or modified reconciliation logic. For the purpose of this capstone, the 'before' state is defined as a workflow model in which document extraction may be automated, but rule creation, node selection, node wiring, variance handling, and deployment remain dependent on technical configuration. This baseline is not presented as a failure of Staple AI's extraction capability. Public sources describe Staple AI as operating document processing and workflow automation at high document volumes and across many languages [1, 2]. The baseline bottleneck is narrower and more specific: non-technical users cannot independently translate reconciliation intent into a deployed, stateful workflow.

This capstone therefore isolates a localized HCI and architecture problem. It does not evaluate whether Staple AI can extract fields from documents. It evaluates whether a non-technical user can instruct an agent to build the reconciliation workflow, verify the generated logic, and publish it to a runtime environment without developer assistance. The test environment is restricted to unstructured PDF and image ingestion because those formats preserve the real ambiguity encountered by finance, warehouse, production, and quality users. Direct EDI feeds, API-native data flows, and ERP master-data redesign are excluded so that the evaluation remains focused on interaction, workflow state, and human comprehension, (capstone scope see appendix 3).

1.3.1. Accounts Payable 3-Way Matching

The accounts payable use case is limited to unstructured PO, goods receipt note, and supplier invoice documents. The grading question is not whether the committee understands 3-way matching. The research question is whether a non-technical AP user can express the rule in plain

language and obtain a functioning, inspectable workflow. A representative instruction is: 'Match invoices against purchase orders and goods receipts. Hold the transaction if quantity or unit price differs beyond tolerance and show me the documents that caused the mismatch.'

Within this use case, Fibula must create a stateful reconciliation path. The generated workflow must bind the PO, GRN, and invoice as one transaction; compare supplier identity, line descriptions, quantities, unit prices, and currency; evaluate tolerance conditions; and pause in a review state when the rule requires human decision. The output is not a payment execution system. It is a payment-ready reconciliation decision with evidence, state history, and audit trace. Full ERP payment execution, bank transfer, and master-data correction are outside scope.

1.3.2. BOM Material Reconciliation

The BOM material use case is limited to comparing master-data component expectations against actual consumption evidence from work orders and goods issue records. The relevant HCI problem is that production variance is rarely self-explanatory. A material issue may exceed the bill-of-material expectation because of approved scrap, rework, wrong unit conversion, incorrect BOM parameters, or floor-level process loss. A stateless pipeline can flag a numeric difference, but it cannot support the operator's decision unless it preserves the work order, component rule, goods issue evidence, tolerance, and approval state.

The expected material consumption and resulting variance are calculated as follows:

$$\begin{aligned} \text{Expected issue quantity} = \\ (\text{BOM component quantity per finished unit} \times \text{confirmed work order output}) + \\ \text{approved scrap allowance} \end{aligned} \quad 1$$

$$\text{Material variance} = \text{actual goods issue quantity} - \text{expected issue quantity} \quad 2$$

The capstone scope is therefore to test whether the user can instruct Fibula to construct this logic without code. The system must generate nodes for document intake, evidence binding, formula evaluation, variance thresholding, and review routing. The user should not manually drag a node canvas or write Python. The chat-first agent should select, configure, and wire the nodes, while the resulting workflow remains visible enough for the user to inspect before publication.

1.3.3. Supplier Non-Conformance Chargebacks

The supplier non-conformance use case is limited to linking quality evidence to financial recovery logic. The input documents include inspection logs, non-conformance records, delivery evidence, supplier identifiers, and proposed debit or chargeback documents. The business problem is not simply extracting a defect category. The system must preserve the chain from received goods to inspection finding to supplier responsibility to financial adjustment. Without this chain, a chargeback becomes difficult to defend when challenged.

The chargeback candidate can be represented as follows:

$$\text{Chargeback candidate} = (\text{rejected quantity} \times \text{agreed recovery rate}) + \text{documented handling or rework cost} \quad 3$$

In scope is the generation of a stateful review workflow that holds disputed cases for human approval and records the decision. Out of scope is direct legal claim management, supplier negotiation, and final ERP posting. The evaluation remains focused on whether a non-technical quality or finance user can create and understand the reconciliation of logic through chat-first interaction.

1.4. Research Objectives: Chat-First Fibula and the 90% No-Help Benchmark

Project Fibula is defined in this capstone as a chat-first, agentic workflow authoring system for stateful document reconciliation. The user does not construct the workflow by manually dragging nodes on a canvas. Instead, the user describes the desired reconciliation behavior in natural language. The AI agent autonomously selects the required node types, configures extraction and matching logic, wires the workflow, and prepares the result for publication to a runtime Document Folder. The canvas is therefore not the primary control surface. It is an inspectable artifact generated by the agent so the user can understand and verify what will run.

The runtime Document Folder is the operational target of the workflow. It is an environment where incoming PDFs or images are processed, matched, held, approved, rejected, corrected, and exported. This distinction matters because a workflow design is not useful until it becomes a deployable runtime process. Fibula's architectural contribution is the linkage between chat-first authoring, generated stateful workflow logic, and runtime document execution. The system must allow an operator to move from intent to deployed reconciliation without requiring a developer to manually translate the request into code or node wiring. (see appendix 4 figure 3)

The primary research objective is to determine whether this chat-first architecture can convert a developer-reliant IDP setup into a self-serve business workflow. The comparison baseline is strict. In the pre-Fibula model, new rule creation requires technical intervention because a developer or implementation specialist must configure the workflow. This is treated as a 100% technical-intervention baseline for new or materially changed reconciliation rules. The capstone must therefore demonstrate a measurable reduction in intervention, not merely showing that an AI agent can generate a plausible diagram.

The quantitative guardrail is the 90% No-Help autonomous benchmark. The threshold is intentionally high because the baseline requires technical help for new rule creation. A marginal improvement would not prove that the system has become self-serve. The capstone therefore requires at least 90% of target non-technical users, in task-based tests, to design, troubleshoot, and publish a reconciliation workflow through plain-language interaction with zero researcher or developer intervention. If a participant requires the researcher to explain a UI element, interpret a generated node, recover an unclear error, or clarify how to publish to the Document Folder, the task is counted as a system failure.

This benchmark aligns the evaluation with the actual HCI problem. The study should measure whether users can state intent, inspect generated workflow logic, understand why a transaction is paused, correct a mismatch, and publish the workflow independently. Correct output alone is insufficient. A participant who completes the task only after external explanation has not validated the self-serve claim. The no-help criterion treats user hesitation, confusion, and intervention requests as design evidence.

Accordingly, the strategic formulation of Section 1 is as follows. Staple AI's capstone need is a localized architecture and HCI problem inside modern document processing: extraction can be strong while workflow authoring, and exception handling remain developer-dependent, stateless, and difficult for non-technical operators to control. Project Fibula addresses this gap by making the chat interface the primary authoring surface, using an agent to generate stateful reconciliation workflows, and publishing those workflows into a runtime Document Folder where documents can be held, approved, resumed, and audited.

2. Literature Review & Industry Technology

2.1 The Evolution of Intelligent Document Processing (IDP)

This section reviews IDP as a three-stage technological progression: first, the digitization of manual document work through OCR and fixed templates; second, the integration of OCR outputs with RPA and AI-enabled invoice automation; and third, the emergence of template-free visual models and LLM-based semantic extraction.

2.1.1 Stage 1: Manual Digitization and Template-Based OCR Architectures

Industrial document processing originally emerged as a response to the operational limits of manual data entry and manual verification. In procurement, production, and finance workflows, transactional and operational documents—such as supplier invoices, purchase orders, goods receipt notes, bills of materials, and goods issue records—serve as business evidence rather than simple text files. They record what was ordered, received, consumed, billed, or disputed. However, when these documents arrive as paper forms, scanned PDFs, or inconsistent image-based files, business users must manually read, enter, compare, and validate the relevant fields. This process is slow, repetitive, and error-prone, especially when enterprises handle large volumes of transactional documents across multiple suppliers and plants. Early Intelligent Document Processing therefore focused on digitization: converting physical or image-based documents into machine-readable text so that downstream systems could store, search, and process document data more efficiently [7], [8].

2.1.2 Stage 2: OCR-RPA Integration and AI-Enabled Invoice Automation

After OCR made transactional documents machine-readable, IDP began to move from text recognition toward process automation. One important step was the combination of OCR with

Robotic Process Automation (RPA). In this model, OCR extracts data from invoice images, while RPA transfers the extracted information into accounting systems, ERP interfaces, or web-based business applications. Rohaime et al. [9] demonstrate this approach through an integrated invoicing solution in which OCR and AI extract semi-structured invoice data, and an RPA bot enters the payment details into the company system. This shows that IDP was no longer limited to reading documents; it started to support the operational handoff between documents and enterprise systems. However, OCR-RPA automation still works best when document fields, task sequences, and system interfaces are relatively stable.

AI and machine learning further extended IDP into accounts payable automation. AP processing involves more than invoice data entry; it includes vendor identification, PO-based line-item matching, accounting code identification, tax code identification, business-rule checking, and approval routing [10]. Tater et al. [10] show that AI-enabled AP systems can be organized as configurable modules across the invoice-processing pipeline. Their production deployment processed around 80,000 invoices, with 76% handled with low or no manual intervention. However, this also shows that AP automation is not a simple extraction task. Because model predictions may be uncertain, such systems still require confidence thresholds, human review, feedback, and adaptation to changing rules or client requirements [10], [11]. Therefore, this stage of IDP improves business automation, but it also reveals the need for workflow-level control when extraction results are used for reconciliation decisions.

2.1.3 Stage 3: Template-Free Visual Models and LLM-Based Semantic Extraction

As invoice layouts became more diverse, IDP research moved beyond fixed templates and coordinate-based extraction. Template-free approaches aim to recognize common business fields even when suppliers change document layouts, rename columns, or arrange line items differently.

Multi-layout invoice research shows that invoices from different suppliers may contain similar fields, such as supplier name, invoice number, total amount, and item lists, but present them in different structures [12]. Layout-aware models therefore preserve spatial information instead of treating OCR output as plain text. For example, Chargrid represents a document page as a two-dimensional character grid, allowing document structure to support information extraction [13]. Later models such as LayoutLM jointly model textual and two-dimensional layout information for document image understanding [14].

Recent LLM-enabled and prompt-guided extraction methods further extend this shift toward semantic entity extraction. Instead of relying only on fixed labels or coordinates, extraction tasks can be specified more semantically, for example by asking the system to identify the final payable amount or the supplier responsible for a variance. This improves flexibility when labels differ across merchants. However, LLM-based extraction still requires careful input representation, prompting, structured output design, and validation [15]. Therefore, modern template-free and semantic extraction methods improve IDP's ability to read variable documents, but they do not remove the need for schema constraints and business-rule validation in reconciliation workflows.

2.1.4 Synthesis: From Extraction-Centric IDP to Stateful Reconciliation Workflows

Taken together, these three stages show that IDP has evolved from manual digitization to process automation and then to more flexible semantic extraction. However, this development remains largely extraction centric. Even when modern systems can read variable document layouts and identify business entities more accurately, reconciliation is not complete once fields are extracted. In AP matching, BOM reconciliation, and supplier chargeback, the system must also preserve document relationships, validation rules, exception states, and user decisions so that each variance can be explained, reviewed, approved, escalated, or audited within the same workflow.

Fibula provides a concrete example of this shift from extraction-centric IDP to stateful reconciliation workflows. Rather than treating document processing as a linear sequence of upload, extract, and export, Fibula places extracted fields inside a workflow context where document relationships, validation rules, exception states, and user decisions can be represented and inspected. In this setting, natural-language interaction helps users express reconciliation intent, while the generated node-based workflow makes the system logic visible before execution.

2.2. Foundational Challenges in Semi-Structured Document Parsing

While Section 2.1 reviewed the evolution of IDP from digitization to semantic extraction, reliable reconciliation requires more than reading document text. Semi-structured industrial documents must be parsed into stable fields, tables, and line-item relationships before they can support matching rules and exception handling. This section therefore examines the main technical friction points in data harvesting: ambiguous field labels, complex table layouts, hierarchical line-item mapping, and the need for structured context prompts to produce predictable extraction outputs.

2.2.1 Ambiguous Field Labels and Schema Inconsistency

Semi-structured documents are difficult to parse because their fields are partially standardized but not consistently represented. Invoices, purchase orders, goods receipt notes, and production-related documents often contain recurring business fields, such as supplier name, document date, invoice number, purchase order number, total amount, tax amount, item code, quantity, and unit of measure. However, these fields may appear under different labels, in different positions, or with different formatting conventions across suppliers and document templates. For example, “Date” may refer to invoice date, delivery date, posting date, or payment due date, while “Amount” may refer to subtotal, tax, discount, or final payable amount. This makes field extraction difficult

because the system must identify not only the visible text, but also the business meaning of the field within the document context.

Prior research on invoice extraction highlights this problem. Saout et al. [8] note that automated invoice processing must handle varying formats and layouts, differences in language and terminology, and possible data inaccuracies. Similarly, Baviskar et al. [12] show that invoices from different suppliers or departments may have unique layouts while still containing common fields such as supplier name, invoice date, invoice number, total invoice amount, and item list. This creates a schema inconsistency problem: the same business concept may be expressed through different labels or positions, while visually similar labels may refer to different concepts. For reconciliation workflows, such ambiguity is critical because a wrongly interpreted header field can break document linkage, distort matching logic, or route an exception incorrectly. Therefore, semi-structured parsing requires schema normalization rather than simple keyword extraction.

2.2.2 Table Layout Complexity and Line-Wrapped Cells

Table fields create a deeper parsing challenge than header fields because they contain repeated line-item records rather than single values. In AP and BOM-related documents, the most important reconciliation data often appears in tables, including item code, item description, quantity, unit of measure, unit price, amount, component item, goods issue quantity, and scrap or rework quantity. However, table layouts vary significantly across suppliers and document types. Column names may differ, column order may change, and the same business field may appear under different headers. Saout et al. [8] emphasize that invoice extraction must address both information extraction and structure recognition, with table processing forming a central challenge in invoice automation.

The difficulty is not only textual but also spatial. Mixed-style tables may contain headers in merged or non-merged cells, and the spatial relationship between a header and its corresponding value may vary across documents [16]. Complex business tables may also include empty cells, dense content, multi-row or multi-column spanning cells, and inconsistent alignment, all of which make cell detection and table structure recognition difficult [17]. Line-wrapped cells create another common problem: a long item description may continue onto the next line, making it unclear whether the second line belongs to the same item or starts a new record. Similarly, summary rows such as “Subtotal” or “Total” can be mistaken for normal line items if the system relies only on visible text. These issues show that table extraction is not simply an OCR task. It requires the system to reconstruct row-column relationships so that each extracted value remains attached to the correct item and business meaning.

2.2.3 Hierarchical Line-Item Mapping Across Documents

Line-item parsing becomes more difficult when reconciliation requires records from multiple documents to be mapped to the same business object. In AP three-way matching, the system must link purchase order, goods receipt, and invoice line items before it can compare ordered, received, and billed quantities. This mapping cannot rely only on row position because the same item may appear in different row orders across documents. It also cannot rely only on exact text matching because product descriptions may be abbreviated, reformatted, or expressed with different units. Maurya et al. [18] highlight this problem in invoice line-item matching, where invoice and purchase-order descriptions may look different at the word level but still refer to the same product. This shows that AP reconciliation requires both document-level linkage, such as PO number, and line-level matching, such as item code, description, quantity, unit of measure, and unit price.

The same issue becomes more hierarchical in BOM reconciliation. Here, the system must connect BOM records, work orders, goods issue records, and scrap or rework logs under a shared production context. The comparison is not only whether two fields match, but whether expected and actual material consumption can be calculated at the correct Work Order + Component Item level. For example, expected consumption may depend on BOM quantity per unit, actual output, and scrap allowance, while actual consumption may combine goods issue quantity with scrap or rework quantity. If a component item is mapped to the wrong row or the wrong work order, the variance calculation becomes unreliable. More broadly, transactional documents often contain nested, document-level, and overlapping relationships that require more than isolated field recognition [19]. Therefore, hierarchical line-item mapping is a core parsing challenge because reconciliation depends on preserving relationships across documents, rows, quantities, and business rules.

2.2.4 Structured Context Prompts for Predictable Extraction

Structured context prompts are important because semi-structured document extraction cannot rely only on unconstrained AI recognition. They guide the model by defining the target field, expected data type, relevant document region, exclusion rules, and relationships between fields. For example, a prompt can specify that the system should extract the final payable amount rather than subtotal or tax, or capture unit price only from the same row as the corresponding item code.

This prompting layer helps convert flexible AI recognition into predictable, schema-aligned outputs that can support AP and BOM reconciliation. Prior work also emphasizes that semi-structured document extraction requires mapping document content into a target schema [19], while LLM-based extraction still depends on prompting, structured output design, and refinement [15]. This leads to the next issue: if extraction depends on schemas, prompts, rules, and exception

states, these elements need to be represented in a workflow structure that users can inspect and control.

2.3. The Cross-Section of Multi-Agent Systems and Visual Directed Graph Networks

While natural-language interfaces lower the barrier for users to express reconciliation intent, they do not by themselves define how the backend should execute field mapping, validation, exception routing, or audit logging. Mission-critical reconciliation therefore requires an architectural interface that translates human intent into deterministic workflow logic. In this context, visual directed graphs and node-based canvases provide a bridge between conversational input and executable backend calculations: they make data mappings, decision branches, and process states visible before the workflow runs.

2.3.1 Natural-Language Intent Capture and Its Limits

Natural-language interfaces are useful because they lower the barrier for non-technical users to express workflow intent. In industrial reconciliation, users may naturally describe tasks in business terms, such as matching invoices against purchase orders and goods receipts, holding payment when quantities differ, or checking whether material consumption exceeds the BOM allowance. This makes conversational interaction attractive as an entry point for workflow creation, especially when users do not know how to manually configure nodes, rules, or data mappings.

However, natural language alone does not define a reliable execution structure. A chatbot response may capture what the user wants, but it does not necessarily show how fields are mapped, which validation rules are applied, what thresholds are used, or where exceptions should be routed. Prior work on human-AI interaction emphasizes that users need to understand system status, capabilities, uncertainty, and available correction paths when working with AI systems [4]. Human-centered

AI research also argues that reliable and trustworthy automation requires both strong computational capability and meaningful human control [20]. Therefore, free-form conversational AI is valuable for capturing reconciliation intent, but it is insufficient as the sole interface for mission-critical supply chain verification.

2.3.2 Visual Directed Graphs as Inspectable Workflow Structures

Visual directed graphs address the main weakness of free-form conversational interfaces by making workflow logic explicit. Instead of leaving the reconciliation logic inside a generated text response, a node-based canvas represents the workflow as connected steps, where each node performs a defined operation and each edge shows data flow, execution order, or conditional routing. This structure is consistent with the broader logic of business process modelling, where process diagrams are used to make business workflows understandable to different stakeholders [5]. In reconciliation settings, this visibility is important because users need to inspect how documents are uploaded, how fields are mapped, which validation rules are applied, and where exceptions are routed.

Industry tools also reflect this architectural pattern. n8n defines workflows as collections of connected nodes, where nodes start workflows, fetch or send data, and process or manipulate data [21]. LangFlow similarly provides a visual authoring environment for building AI-powered agents and workflows [22]. These platforms show why visual automation canvases are useful: they make the relationship between actions, tools, and data transformations visible. For Fibula-like reconciliation systems, this matters because the canvas is not only a design surface; it is also a way for users to verify whether natural-language intent has been translated into the correct workflow structure before execution.

2.3.3 Deterministic Node Structures for Stateful Reconciliation

Mission-critical supply chain verification requires deterministic node structures because reconciliation outcomes affect payment release, material variance reporting, supplier responsibility, and audit records. In AP matching, for example, the system must compare purchase orders, goods receipts, and invoices before deciding whether an invoice can be paid or should be held. In BOM reconciliation, the system must compare expected material consumption with actual goods issue, scrap, or rework records. These tasks cannot depend only on a conversational response, because each decision must be based on explicit fields, rules, thresholds, and traceable document relationships.

Deterministic nodes provide this control. Data Mapper nodes can normalize fields such as supplier/vendor name, item code, quantity, unit of measure, currency, and document date. IF/SWITCH nodes can apply explicit conditions, such as quantity mismatch, unit price variance, date-window checks, or tolerance thresholds. Reconciliation Engine nodes can execute structured comparisons, such as PO-GRN-Invoice matching or BOM-GIR-Scrap comparison. These nodes make backend calculation visible as workflow logic rather than hidden model behavior. This is consistent with enterprise AP automation research, where invoice processing is treated as a multi-step pipeline involving vendor identification, line-item matching, rule checking, thresholds, and feedback [10].

More importantly, deterministic workflow nodes help maintain active document session parameters. These parameters include the documents being matched, extracted fields, primary keys, validation rules, exception status, user decisions, and the next execution path. Without such state, a system may identify a mismatch but fail to preserve why it occurred, which rule triggered it, or how the case should continue after review. Therefore, the role of visual directed graphs is not only

to make automation easier to build, but also to preserve the state and logic required for safe reconciliation. This completes the bridge from human intent capture to deterministic backend calculation.

Chapter 3 research methodology

This chapter will explain the methodology used to evaluate Fibula as an AI-enabled workflow automation platform for industrial reconciliation. This involves decomposition of reconciliation processes and human-centered usability testing. First, the team translated AP and BOM reconciliation requirements into workflow tasks, matching keys, and validation checkpoints; next, users were observed while using Fibula through task-based testing and think-aloud observations. Finally, UX mapping tools were used to document users' journeys and interactions before synthesizing the findings in chapter 4.

3.1 Industrial cross document verification logic

Before conducting the user research, it was the team's responsibility to decompose industrial processes into workflow-level verification steps. Although Fibula contains multiple workflows, from bank statement reconciliation to sales order processing, the team mainly focused on decomposing AP and BOM reconciliation. (see appendix 9 and 10 for matching logic)

For the AP reconciliation use case, the business logic was translated into a Fibula workflow consisting of four main operational stages: document ingestion, schema-based extraction, validation checking, and exception routing. The process began with a Manual Upload node, where users uploaded invoice-related documents. These documents were then passed to an Invoice Extraction schema node, which performed structured extraction of invoice fields. The extracted

data then entered a Validation Check node, the operational control, which included standardizing extracted text fields, applying case-insensitive matching to supplier names and item identifiers, normalizing date and currency formats, checking whether invoice dates fell within the accepted date range, and comparing monetary values within a defined tolerance. Depending on the outcome, the record will be routed either to approval or review. (see appendix 5)

For BOM reconciliation, the logic was decomposed into expected consumption, actual consumption, and variance routing. The workflow required the system to connect Work Orders, BOM or Item Master records, Goods Issue Records, and Scrap/Rework Logs through Work Order ID and item/component identifiers. The expected consumption was calculated from the standard BOM quantity per unit and the actual production output, while the actual consumption was derived from the GIR/backflush quantity and scrap/rework quantities. The comparison between expected and actual consumption formed the basis for identifying material variance and routing the result for follow-up action.

3.2 User research: semi-structured interviews and observations

After the reconciliation logic had been translated into Fibula's node-based workflow, the team's next objective would be to evaluate whether the user could operate the workflow independently. The users were tested to determine whether they could understand the purpose of AI agents, configure workflow steps, interpret validation output, and decide whether the result requires review. It is essential to conduct such testing because enterprise workflow automation succeeds only when users can complete tasks effectively, not merely when the underlying extraction model performs accurately. A total of 15 participants took part in the user research. Participants were selected primarily from non-technical or semi-technical backgrounds, aligning with Fibula's

objective of making enterprise workflow automation accessible to users who do not write code or understand backend workflow configuration. The research was conducted on Fibula, Staple AI's workflow automation environment. The live application was accessed through Fibula's web app, and all participants provided consent to participate in the study. (appendix 6)

The study adopted a semi-structured, task-based usability testing approach. The team is prepared with a set of tasks that participants should perform during the session (such as AP reconciliation), but participants are encouraged to test whichever workflow they prefer. The testing structure was semi-structured because the core mission and product areas were consistent across sessions, while participants were still allowed to explore naturally when they encountered unexpected features, questions, or blockers. This design reflected the exploratory nature of Fibula, where users may enter through the AI agent, template builder, document schema interface, reconciliation logic, or integration settings.

Table 1 Research Methodology

Method component	How it is applied	Why is it suitable
Tasked based usability testing	Users attempted realistic Fibula workflow tasks	Evaluated whether users could complete real operational actions
Semi-structured questioning	Researchers used prepared questions but followed user behavior	Allowed consistent comparison while capturing unexpected friction
Observation logging	Researchers recorded actions, blockers, and comments	Captured usability issues that users may not report directly
Post-task probing	Researchers asked clarifying questions after friction appeared	Helped explain why users misunderstood certain workflow steps

To maximize the value of insights that are gained from the research and imitate a real-world work environment, there are a number of protocols and rules the team followed. Starting with the think-aloud protocols, the think-aloud protocol is when participants verbalize their thoughts while performing a task, giving insight into actions, reactions, and expectations. Participants should describe expectations and feelings, actions. In this research, participants were instructed to verbalize what they were looking for, what they expected to see, and how they interpreted the workflow. This method was particularly suitable because many usability issues were cognitive rather than purely visual. For example, users could click through the interface but still misunderstand whether the AI had completed a validation check. However, the team recognized that think-aloud can slightly affect task behavior because verbalizing thoughts may add cognitive load. For this reason, researchers avoided over-prompting participants and used short reminders, such as “what task are you performing right now,” only when users remained silent for too long.

The study applied a limited-intervention observation rule, in which the team did not immediately help participants when they hesitated, chose the wrong path, or misunderstood. Instead, participants were allowed to explore and attempt independently. If a participant failed to progress after three unsuccessful attempts, the incident was recorded as a usability failure before the researcher offered a minimal nudge. The nudge was framed as a guiding question rather than a direct instruction, such as “Where would you expect to find the result?” This approach allowed the team to distinguish between minor confusion. This rule was particularly relevant because it corresponds to Fibula’s product goal: creating a workflow for non-technical users. If a user could only complete the workflow after repeated explanations by researchers, the platform had not yet achieved the intended self-service standard.

After each session, the team converted raw observations into an interview snapshot. Each snapshot separated direct observations from interpreted insights and improvement opportunities. Observations were then coded into recurring friction categories. This evidence-based coding process prevented the team from treating isolated comments or personal design preferences as product requirements. A design issue was elevated only when it recurred or blocked task completion. The research process followed an iterative cycle. When a problem was discovered through an interview, it would be discussed with the team and Staple AI. The next round of testing then examined whether similar blockers persisted, or the changes reduced friction. This created a continuous loop from observation to diagnosis, product discussion, improvement, and retesting. (Interview snapshots see appendix 7)

3.3 Task journey and recording

Because participants entered Fibula through different paths and sometimes selected different workflow types, the team did not assume a single identical user journey. Instead, we developed a reference evaluation journey to normalize observations across sessions. This journey represents the common stages required to create, test, and interpret a Fibula workflow, regardless of whether the user started from the chatbot, the builder, or a template.

Where possible, the team also conducted follow-up checks with previous participants or later test users to validate whether the revised interaction reduced the originally observed friction. These follow-up checks were treated as qualitative usability validation rather than controlled statistical remeasurement, because the product was still evolving during the sprint. (process table see appendix 8)

Overall, this chapter established the methodological foundation for evaluating Fibula as both a reconciliation workflow platform and a human-AI interaction system. By translating AP and BOM reconciliation logic into platform-level workflow tasks, the study ensured that user testing was grounded in realistic industrial use cases. The combination of task-based testing, think-aloud observation, limited-intervention no-help protocol, interview snapshots, and reference journey recording created a structured evidence chain for the diagnostic analysis in Chapter 4.

4. Diagnostic Findings & Systemic UX Analysis

This chapter presents the main UX findings from the Fibula usability testing process. Due to the internal and unpublished nature of Fibula, the company was unable to provide full production-level extraction-accuracy data, validation success rates, or large-scale document-processing logs for this capstone. Therefore, the empirical evidence in this report is based on usability validation rather than on model performance benchmarking. The main question is whether non-technical users can create, inspect, test, and understand a reconciliation workflow without assistance from researchers or developers.

This distinction is important because Fibula is designed as a chat-first workflow authoring system. Users describe their reconciliation intent in natural language. The AI agent then creates a node-based workflow. The canvas helps users inspect the generated logic, while the runtime Document Folder processes documents and holds cases for approval, rejection, correction, or review.

The tests showed that Fibula already provides useful visual feedback. Users can see the pipeline running. They can also see the current processing step during execution. Therefore, the main issue is not total pipeline invisibility. The issue appears when the workflow pauses, fails validation, or

enters an exception state. At that moment, many users cannot understand why the process stopped, where the relevant evidence is, or what action to take next.

The findings are based on task-based usability testing, think-aloud observation, limited-intervention testing, and interview snapshots. The no-help rule was used as the main evaluation standard. If a participant needed the researcher to explain a UI element, interpret a generated node, or recover an error; the task was counted as a usability failure. This standard aligns with Fibula’s goal of enabling self-service workflow creation for non-technical users.

4.1. Quantitative Diagnostic Metrics & Journey-Level Friction Analysis

The team used a severity index to classify the user's friction observed during unguided tasks. (diagnostic metric and severity index see appendix 11, table 2) The measured task failure rate does not mean the backend crash rate. It refers to the percentage of observed users who became blocked, chose the wrong path, misunderstood the system's state, or needed outside help at a specific journey stage.

Table 3 the main diagnostic findings across the user journey

Journey Stage	Identified Interaction Risk	Severity Index	Measured Task Failure Rate %	Root UI Root Cause Breakdown
1. Start	Hidden platform entry points	P2	42%	Slow onboarding caused by flat, non-suggestive welcome screens.

2. Template	Excessive, unstructured choice arrays	P1	58%	Choice overload driving users to choose incorrect system templates.
3. Configure	Ambiguous node schema fields	P0	73%	Cognitive load due to complex keyword requirement
4. Test	Hidden pipeline execution states	P0	67%	Total loss of system trust caused by silent background AI updates.
5. Output	Obscured document exception panels	P2	51%	Processing errors are ignored due to hidden data variance logs.

The highest failure rate appeared in the configuration stage. This was a P0 issue because users could not always verify whether the AI-generated workflow matched their business intent. Many users understood the business rule, such as matching invoices against purchase orders and goods receipt notes, or checking material consumption against a BOM. However, they could not easily connect that rule to schema fields, matching keys, keyword pairs, validation nodes, and exception routes. The canvas was visible, but its logic was not always readable in business terms. This reduced the value of the canvas as an inspectable artifact.

The test stage also produced a P0 failure rate. This result needs careful interpretation. Users could see the pipeline progressing and could locate the paused node. In addition, the interface displayed a "What's Next" prompt after a pause. The issue was that this prompt primarily provided direction for the next action. It did not always show the exact root cause, the affected field, or the specific document that required correction or re-upload. As a result, users still had to open the node details, inspect the internal outputs, and manually identify the point of error before they could recover.

The output stage showed a different but related problem. Variance evidence was not absent. The system could provide information about the location of a difference or the document evidence linked to it. However, this information often required users to open a deeper panel. The main processing interface also contained many fields, status messages, tables, and exception details. For non-technical users, this high information density made it hard to know which item mattered most and which action should follow.

Overall, the journey-level metrics show a pattern of systemic friction rather than isolated mistakes. The strongest barriers appeared when users had to understand generated logic, diagnose a paused pipeline, or interpret variance evidence. These findings support deeper behavioral analysis in the next section.

4.2. Deep Synthesis of Core Behavioral Findings

The journey-level data can be grouped into three core behavioral findings. These findings follow the original diagnostic outline: spatial desynchronization, convenience bias and choice of fatigue, and operational opacity with interaction dead-ends. Each finding describes a different way in which Fibula's visible workflow structure did not always translate into user confidence or independent recovery.

4.2.1. Spatial Desynchronization and the Perceived AI Black Box

The first finding is spatial desynchronization. This does not simply mean that the interface had several pages or panels. It means that the user's natural-language intent, the AI-generated workflow, the canvas structure, and the runtime state were not always connected in one continuous explanation. The system may have been connected internally, but the user did not always experience it as one clear task flow.

Fibula's intended interaction model is chat-first. Users describe the reconciliation task in the chatbot. The AI agent creates the workflow. The canvas then shows the generated nodes and connections. During testing, however, users often saw the generated canvas without being able to keep the chatbot explanation beside it. If they wanted to check what they had asked for, or how the AI explained the generated workflow, they had to return to the chatbot page.

This produced a strong black-box feeling. The problem was not that users saw nothing. In fact, they could see the workflow structure. The problem was that they saw the result without fully understanding the generation logic behind it. Users could not always tell why the AI chose these nodes, why the nodes were connected in this order, or how each node represented the original business rule. A visible workflow therefore still felt opaque.

This psychological effect is important for trust. A user may ask Fibula to perform AP three-way matching, BOM reconciliation, or exception routing. If the generated nodes appear on the canvas but the user cannot connect them back to the original prompt, the user cannot judge whether the AI has understood the request correctly. Even when the workflow is technically correct, the lack of explanation can make the system feel unpredictable.

The black box feeling therefore came from the gap between seeing generated nodes and understanding why the AI agent created, connected, and configured those nodes in that specific way. Spatial desynchronization made this gap larger because users had to move between the chatbot, canvas, pipeline status, and exception panels to reconstruct the full logic. This finding explains why a node-based canvas alone is not enough. The canvas must be paired with context and explanation.

4.2.2. Convenience Bias and Choice Fatigue

The second finding is convenience bias and choice fatigue. Fibula offers templates, workflow options, and configuration choices. These options are useful because different reconciliation tasks require different logic. However, too many visible choices can create a burden for non-technical users. When users encounter dense lists and unfamiliar labels, they often choose the easiest path rather than the most accurate one.

This behavior was not caused by carelessness. It was a rational response to high cognitive loads. Many users did not know which option controlled the business logic and which option was only a supporting setting. As a result, users tended to select the default template, the shortest option, or the option that looked most familiar. This reduced effort in the moment, but it could move the workflow away from the intended task.

The template stage failure rate shows why these matters. A user who wants AP three-way matching may accidentally start from a general invoice extraction template. A user who wants BOM variance checking may select a general document processing flow instead of a reconciliation flow. The workflow may still run, but it may not include the correct matching keys, tolerance rules, evidence binding, or exception routing.

The same issue appeared in custom configuration. When users had to type custom properties or adjust schema fields, some avoided manual input because they were unsure what each field meant. They preferred predefined options because those options felt safer. However, predefined options are only helpful when they match the user's business intent. If they are too broad or too similar, they increase the risk of a wrong starting point.

This finding shows a tension in Fibula's interface. The product aims to let users express work in natural language, but long option arrays push users back into a traditional enterprise software experience. Users have to scan, compare, and choose technical labels before they fully understand the workflow. The diagnostic conclusion is not that templates should be removed. The conclusion is that template selection needs stronger guidance and better grouping by business intent.

4.2.3. Operational Opacity and Node-Level Recovery Dead-Ends

The third finding is operational opacity with node-level recovery dead-ends. This finding must be defined carefully. It does not mean users could not see the pipeline. Users can see the current processing step. When the workflow pauses, they could usually locate the paused node. The interface also provided a What's Next prompt to suggest the next action. The opacity appeared after this point, when users had to diagnose the exact cause of the pause.

What's Next prompt helped users understand that a next action existed. However, it did not always provide root-cause diagnosis. Users still needed to know why the node paused, which field caused the issue, etc. This is the difference between action-level guidance and diagnosis-level guidance. If users chose to handle the issue manually, they had to open the paused node, inspect the node details, read internal outputs, and identify the specific error point. This required them to understand node

logic, schema fields, matching rules, and document evidence. For developers or experienced implementation users, this process may be reasonable. For non-technical users, it became a high-friction diagnostic task.

Users also had another path: they could return to the chatbot and ask it to explain or handle the problem. This was useful, but it created cross-panel context loss. The user had to leave the canvas and the paused node, move back to the chatbot interface, and then reconnect the chatbot response with the node state. This made recovery possible at the system level, but difficult at the interaction level.

For this reason, a recoverable pause could still become a no-help failure. The system showed the paused node and suggested a next step, but users still had to perform manual root-cause analysis. The main issue was not missing progress visibility. It was the gap between visible pause state, What's Next guidance, and direct root-cause recovery.

4.3. Interface Backstage Processing Gaps

The behavioral findings above point to a deeper backstage processing gap. Fibula may store workflow state, node outputs, exception status, and variance evidence. However, the interface does not always translate these backend states into business-level guidance at the moment when the user needs it. This gap explains why users could see parts of the process but still struggle to complete it independently.

4.3.1. Engineering Assumptions vs Operator Mental Models

From an engineering view, a workflow system naturally contains many internal states. A node can wait for input. A schema can fail validation. A document can miss a required field. A pipeline can

pause. A runtime process can retry. A log or node output can show a technical reason for failure. These are normal backend behaviors.

Non-technical operators think differently. They think in business tasks. They ask whether the system read the correct documents, whether the PO, GRN, and invoice match, whether actual material consumption exceeds expected consumption, which field is outside tolerance, and whether the case should be approved, rejected, corrected, or reviewed. They do not naturally think in node states, schema errors, or runtime logs.

This difference creates trust risk. A pause may not be a system failure. It may mean the workflow is waiting for review, a variance exceeded the rule, or a field needs correction. However, if the interface presents the pause mainly as a technical state, users may interpret it as failure. This is especially serious in reconciliation because the output can affect payment, material variance reporting, supplier responsibility, and audit confidence.

4.3.2. Developer-Oriented Signals as Partial Clues, Not User Guidance

Many interface signals worked more like developer clues than user guidance. A developer clue helps a technical person diagnose the system. It can be short and based on internal logic. User guidance is different. It should explain the issue in plain language, show the affected document or field, and suggest the next action.

For example, “validation failed” is not enough for a business user. The user needs to know whether the failure came from a missing goods receipt note, a quantity difference above tolerance, a unit of measure mismatch, or an item code mapping issue. “Schema mismatch” is also not enough. It

should be translated into a concrete message, such as “invoice date was not found” or “item code could not be matched to the purchase order.”

This gap explains why users still needed help even when the system showed status or node information. The platform contained useful diagnostic clues, but the user had to convert those clues into operational actions. Under the no-help rule, this conversion burden counts as a usability failure because the system has not yet fully supported self-service troubleshooting.

4.3.3. Buried Variance Evidence and High-Density Exception Panels

The final backstage gap concerns variance evidence. The issue was not evidence of absence. Fibula could provide variance location and related document information. The issue was evidence of discovery and evidence of actionability. Important information was sometimes placed inside a deeper detail panel, exception panel, or comparison view. Users had to know where to open the next layer before they could understand the problem.

High information density made this harder. The processing interface could show many fields, statuses, tables, node outputs, and error messages at the same time. More information did not always mean clearer information. Without strong visual priority, users could miss the specific difference field, overlook the review state, or fail to notice the action required.

Overall, our analysis shows that Fibula’s central UX issue is not total invisibility. The pipeline can show progress. The paused node can be located. What’s Next can provide a next-step direction. The deeper issue is that users still need clearer root-cause explanation, better evidence surfacing, and smoother movement between chatbot, canvas, node details, and runtime state.

5. Engineering Evolution & Recommendations

5.1. Concrete Platform Implementations (Before vs. After Analysis)

The engineering evolution of Fibula during the capstone can be summarized as a shift from extraction-centered automation to stateful reconciliation authoring. The baseline platform could represent document-processing logic, but the interaction still made non-technical users feel as if important workflow activity was hidden behind the interface. For an AP, warehouse, production, or quality user, this was not a cosmetic inconvenience. Reconciliation decisions depend on a visible chain of documents, extracted fields, rule parameters, variance reasons, review actions, and audit records. When that chain is not visible, the user cannot judge whether the workflow reflects the original business intent.

The first implementation addressed spatial desynchronization. In the earlier interaction model, the user entered a natural-language instruction and later received a completed graph, but the canvas did not show how the agent was constructing the workflow while the task was running. This delay created a black-box effect: the user could not see whether PO, GRN, invoice, BOM, GIR, or scrap/rework evidence was being bound correctly, or whether the correct matching and exception nodes were being created. The optimized design introduced WebSocket-driven live canvas synchronization. Nodes are rendered, connected, and updated while the agent processes the prompt, so the canvas becomes a live state display rather than a post-generation artifact. This improves trust because the user can observe intake, mapping, comparison, and review states as they emerge. The second implementation addressed convenience bias and choice of fatigue. The baseline interface exposed long option arrays and dense text summaries. In testing, this encouraged users

to select default paths even when the workflow required a custom tolerance, date window, UOM mapping, or review condition. The redesigned interaction reduced verbose choices to two or three predictive chips plus a fallback path for custom configuration. The point was not simply shortening text. Each chip represents a likely next workflow state, such as reviewing matched fields, adding a tolerance rule, or publishing to the document folder. This lowers the decision burden while preserving the ability to define non-standard reconciliation logic.

The third implementation addressed operational opacity. Silent backend pauses and raw infrastructure messages caused users to interpret recoverable system states as failures. In document reconciliation, this is especially harmful because the operator must remain confident that the document chain has not been lost. The optimized design replaced technical log strings with natural-language advice cards. These cards explain what happened, why it matters, what evidence or field should be checked, and which recovery action is available. For example, a field-mapping issue can be presented as an uncertain relationship between a source label and a canonical field, with options to correct the mapping, rerun extraction, or route the case to review. This turns a technical exception into a business-understandable recovery state.

The fourth implementation addressed layout rigidity. The project evidence included BOM tables, GIR samples, scrap/rework logs, reconciliation diagrams, and scanned or photographed operational forms. These artifacts require spatial inspection: users need to compare a source document, an extracted table, a node graph, and a conversational instruction without losing context. Fixed panels made that difficult. Draggable split panes and table expansion rules allowed users to allocate more space to the document, table, or workflow graph depending on the task. This matters because field visibility is part of auditability. A user should not accept a matched or unmatched decision merely because the supporting evidence is hard to inspect.

Table 4: Before-and-after comparison

Section 4 Baseline Issue	Baseline Evidence	Section 5 Improvement	Follow-up Validation Signal
Entry points and upload actions were not visible enough for first-time users	42% task failure at start stage; users hesitated at the welcome page, missed upload triggers, or depended on chat guidance to find where to begin	Guided onboarding flow, clearer upload entry points, sidebar/tooltips, and direct navigation between chat and workflow pages	Follow-up users moved into the workflow more smoothly because the starting point, upload action, and navigation path were easier to recognize.
Template and setup choices encouraged default or shallow selections	58% task failure at template stage; users skipped long descriptions, chose default options, or questioned why many similar options were shown	Shorter agent summaries, 2-3 predictive choices, custom fallback, and template search/keyword support	Follow-up users indicated that the simplified setup flow made choices feel more manageable and reduced the tendency to select defaults only for speed.
Node configuration and workflow logic were hard to interpret or modify confidently	73% task failure at configuration stage; users struggled to understand node settings, workflow state, or document-to-node relationships	Live workflow synchronization, clearer node-state visibility, document-ingestion review, and contextual tooltips	Follow-up users reported that clearer node status and document linkage made the generated workflow easier to understand and less risky to adjust.

Testing states and recovery actions were too hidden	67% task failure at testing stage; users encountered unclear upload failures, buried rerun controls, authentication stalls, or technical error states	Rerun option inside the test modal, clearer file-type handling, fixed image rendering, improved Google integration, and visible recovery prompts	Follow-up users treated testing pauses as recoverable steps because rerun, upload, and correction actions were more visible at the moment of failure.
Output and reconciliation evidence were difficult to verify or act on	51% task failure at output stage; users could not easily locate results, verify dummy-variable outputs, or inspect reconciliation evidence	Intermediate result visibility, document detail download, zoom controls, table-panel expansion, and clearer review/output guidance	Follow-up users found it easier to confirm the reconciliation result because matched/unmatched evidence and the next action were presented more clearly.

Together, these changes show that the engineering issue was not only model capability. The platform needed to expose enough workflow state for a non-technical user to author, inspect, troubleshoot, and publish a reconciliation workflow without hidden expert support. The improvements therefore connect directly to the 90% No-Help benchmark: a workflow is not self-serve if it runs only after a researcher or developer explains what the interface means.

5.2. Tactical Recommendations for Extended UX Capabilities

The next tactical improvements should continue with the same principle: reduce the distance between document evidence, workflow state, and user action. First, Fibula should extend split-pane behavior into a richer document viewport layer. Industrial records are often scans, PDFs,

photographed forms, handwritten logs, bilingual tables, or supplier-specific templates. A zoom-to-field tool should connect each extracted value to the source region in the document. Hovering over a PO number, item code, quantity, tax value, defect code, or scrap quantity should highlight the evidence location; selecting a document region should show the extracted value and the downstream node that uses it. This would strengthen both trust and auditability.

Second, the system should provide pre-ingestion variance summaries. Before a workflow is published or tested, the interface should state which documents are required, which fields are mandatory, which matching keys will be used, and what variance checks will be applied. An AP preview might describe PO-GRN-invoice matching, UOM normalization, receipt-to-invoice date checks, and payment hold rules. A BOM preview might describe expected consumption, GIR or backflush evidence, scrap/rework adjustments, and allowed variance thresholds. Explaining the intended logic before execution gives users a chance to correct assumptions before a confusing output appears.

Third, Fibula should add fuzzy semantic field mapping for vendor and plant-specific documents. Manufacturing records vary in naming conventions and layout. Labels such as Item Code, Part No., Material, Qty Issued, GI Qty, and Backflush Usage may refer to equivalent canonical fields depending on context. The system should propose mappings with confidence scores, show the source label and canonical field, and allow the user to accept, correct, or save the mapping. The important governance point is that fuzzy inference must remain inspectable; it should assist configuration without silently rewriting the business rule.

Fourth, recovery flows should remain within the same task context. If a workflow test fails because a document class is missing, a field is ambiguous, or a tolerance rule is underspecified, the user

should be able to upload missing evidence, adjust the rule, correct the mapping, rerun extraction, or route the case to manual review from the same modal. Finally, output review should become role sensitive. AP, warehouse, production, quality, and finance users need the same evidence chain, but they interpret it through different operational actions. The interface should therefore present matched status, variance reason, source documents, next action, and audit trace in language appropriate to the user's role.

These recommendations should be prioritized by user-task evidence rather than feature volume. A new feature should move forward only if it helps a representative user complete authoring, inspection, testing, or recovery without external explanation. If it adds configuration burden without reducing no-help failures, it should be deferred.

5.3. Strategic Horizon Planning: The Autonomous Agentic Roadmap

The strategic roadmap should be framed as controlled progress from supervised workflow authoring toward bounded autonomous reconciliation. The capstone does not justify immediate unsupervised financial execution. Its contribution is a stateful human-in-the-loop model: documents are ingested, evidence is bound, variance logic is applied, disputed cases are held, and user decisions are recorded. Future autonomy should expand only where auditability, explainability, confidence thresholds, and human override remain intact.

In the near horizon, Fibula should become a stronger supervised reconciliation assistant. AP nodes should distinguish quantity, price, tax, freight, missing receipt evidence, and receipt-date exceptions. BOM nodes should distinguish approved scrap, rework usage, UOM uncertainty, missing GIR evidence, and suspected supplier-related loss. The goal is not merely to calculate a

delta, but to explain what the variance means operationally and preserve the evidence chain behind the recommendation.

The next horizon is semi-autonomous exception triage. Autonomous nodes can detect discrepancies, classify likely causes, and prepare recommended actions for review. AP workflows could recommend approval within tolerance, holding, requesting vendor clarification, or routing to finance. BOM workflows could separate normal scrap allowance from process loss or suspected supplier non-conformance. Chargeback workflows could assemble inspection records, rework logs, defect codes, and debit-note candidates into a structured recovery package.

The longer horizon is an autonomous agentic network composed of specialized intake, field-mapping, matching, variance, audit-trace, and dispute-document agents. These nodes could monitor incoming document folders, bind evidence into case objects, cross-reference inventory and finance records, and generate exception packets. However, payment execution, legal claims, supplier negotiation, final ERP posting, and master data correction should remain governed by approval points. A practical 12–24-month roadmap should therefore harden self-serve authoring first, expand supervised reconciliation intelligence second, and introduce autonomous exception orchestration only for cases that remain within policy and confidence thresholds.

This roadmap should also keep a clear distinction between automation and accountability. As Fibula becomes more agentic, every node that changes a reconciliation decision should preserve the source documents, extracted fields, mapping assumptions, rule version, confidence state, user action, and downstream consequence. These elements are not administrative overhead; they are the controls that allow finance, production, warehouse, and quality teams to trust automation without surrendering responsibility. A mature workflow should therefore make autonomy reversible and auditable. Users should be able to inspect why a case was held, why a variance was

accepted, which evidence supported a recommendation, and how the workflow can be corrected if a document layout, vendor naming convention, or policy threshold changes. In this sense the long-term value of Fibula is not only faster for document processing but also a more governable operating model for industrial reconciliation.

The roadmap should also define how success will be measured after these phases are implemented. For a stateful reconciliation platform, success should not be limited to extraction accuracy or the number of automated documents processed. More relevant indicators include the proportion of workflows authored without support, the number of exceptions resolved without leaving the workspace, the share of variance cases with complete evidence links, and the time required for a user to understand why a workflow paused. These measures keep the roadmap aligned with the project thesis: operational autonomy depends on visible state, not only backend intelligence.

A second strategic requirement is governance of workflow change. Manufacturing and supply-chain documents change frequently as vendors alter templates; plants modify material codes, finance teams adjust tolerances, or quality teams introduce new chargeback categories. Fibula should therefore treat workflow versions, mapping decisions, and tolerance parameters as controlled objects. Each change should be visible, reversible, and tied to the user or role that is approved by it. This would allow the platform to support continuous improvement without turning evolving business logic into another hidden technical dependency.

6. Project Governance & Critical Reflection

6.1. Strategic Project Management & System Boundaries

Project Fibula was governed as an industry capstone with a fixed academic timeline, a real sponsor environment, and a cross-disciplinary problem space. The team used scope control to keep the

work focused on one testable question: can a non-technical operator describe, inspect, test, and publish a stateful reconciliation workflow without researcher or developer help. The 8-week evaluation cycle and 90% no-help rule converted observed user friction into engineering requirements. The team deliberately prioritized PDFs, scans, BOM images, GIR samples, and scrap/rework logs, while deferring ERP/EMS integration, EDI feeds, relational databases, and CSV/XLSX pipelines so that interface findings would not be mixed with backend integration complexity.

6.2. Team Contribution and Gantt chart

During this capstone project, the team ensured everybody contributed as equally as possible, unlike the first half of the project, where everybody took on different tasks and pieced them together. For the user research and insights exploration, every member was encouraged to follow the same methodology to ensure the controlled environment. Before the conclusion of this project, Zhang Zhiyuan filmed the video of the group’s work, which will be presented in the showcase alongside a website introducing the project and Fibula, created by Mo Yuyang and Chen Jung-Lu.

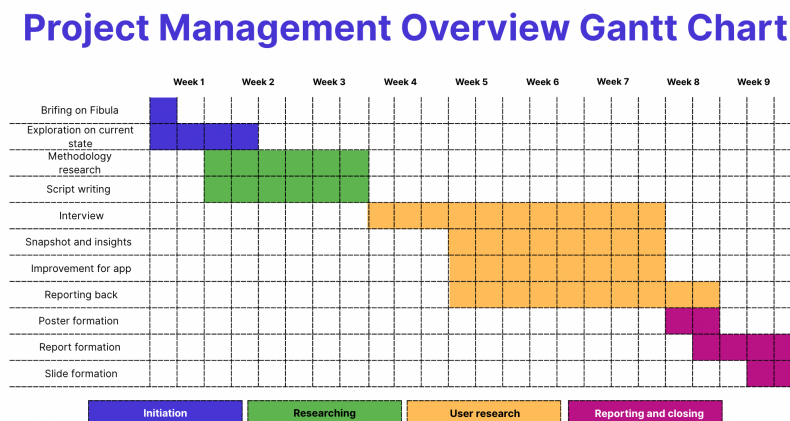


Figure 5. Project Timeline

6.3. Conclusion

This project evaluated how Fibula can move industrial document processing beyond AI extraction toward stateful, autonomous reconciliation workflows. By analyzing AP, BOM, and supplier non-conformance use cases, the team showed that the core challenge is not only extracting data accurately but also helping users understand document relationships, validation logic, exception states, and next actions.

Through task-based user testing, think-aloud observation, no-help evaluation, and interview snapshots, the study identified three main adoption barriers: fragmented interaction between chat and canvas, choice fatigue during setup, and unclear recovery when workflows pause or produce variance results. These findings showed that non-technical users need more than a generated workflow; they need visible logic, clear guidance, and actionable outputs.

Overall, Fibula has strong potential to support self-service reconciliation for non-technical operations teams. Future improvements should focus on clearer workflow explanations, simpler configuration, stronger root-cause guidance, and more visible matched/unmatched results to increase trust and reduce manual support.

References & Supplementary Appendices

References:

- [1] Staple AI. Document processing regulators can verify. <https://www.staple.ai/>
- [2] Google Cloud. Staple AI processes tens of millions of documents with near-100% accuracy using Google AI. <https://cloud.google.com/customers/staple-ai>
- [3] National Institute of Standards and Technology. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0). <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-ai-rmf-10>
- [4] Amershi, S., et al. (2019). Guidelines for Human-AI Interaction. CHI 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
- [5] Object Management Group. (2014). Business Process Model and Notation (BPMN), Version 2.0.2. <https://www.omg.org/spec/BPMN/2.0.2/>
- [6] Object Management Group. (2017). Unified Modeling Language (UML), Version 2.5.1. <https://www.omg.org/spec/UML/2.5.1/>
- [7] Baviskar, D., Ahirrao, S., Potdar, V., & Kotecha, K. (2021). *Efficient automated processing of the unstructured documents using artificial intelligence: A systematic literature review and future directions*. IEEE Access, 9, 72894–72936.
- [8] Saout, T., Lardeux, F., & Saubion, F. (2024). *An overview of data extraction from invoices*. IEEE Access, 12, 19872–19886.
- [9] Rohaime, N. A., Abdul Razak, N. I., Thamrin, N. M., & Shyan, C. W. (2022). *Integrated invoicing solution: A robotic process automation with AI and OCR approach*. 2022 IEEE 20th Student Conference on Research and Development (SCORED), 30–33. <https://doi.org/10.1109/SCORED57082.2022.9973841>
- [10] Tater, T., Gantayat, N., Dechu, S., Jagirdar, H., Rawat, H., Guptha, M., Gupta, S., Strak, L., Kiran, S., & Narayanan, S. (2022). *AI driven accounts payable transformation*. Proceedings of the AAAI Conference on Artificial Intelligence, 36(11), 12405–12413.
- [11] Dragomirescu, O.-A., Crăciun, P.-C., & Bologa, A. R. (2025). *Enhancing invoice processing automation through the integration of DevOps methodologies and machine learning*. Systems, 13(2), 87. <https://doi.org/10.3390/systems13020087>
- [12] Baviskar, D., Ahirrao, S., & Kotecha, K. (2021). *Multi-layout unstructured invoice documents dataset: A dataset for template-free invoice processing and its evaluation using AI approaches*. IEEE Access, 9, 101494–101512. <https://doi.org/10.1109/ACCESS.2021.3096739>
- [13] Katti, A. R., Reisswig, C., Guder, C., Brarda, S., Bickel, S., Höhne, J., & Faddoul, J. B. (2018). *Chargrid: Towards understanding 2D documents*. Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, 4459–4469. <https://doi.org/10.18653/v1/D18-1476>
- [14] Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., & Zhou, M. (2020). *LayoutLM: Pre-training of text and layout for document image understanding*. Proceedings of the 26th ACM SIGKDD International

Conference on Knowledge Discovery & Data Mining, 1192–1200.

<https://doi.org/10.1145/3394486.3403172>

[15] Colakoglu, G., Solmaz, G., & Fürst, J. (2025). *Problem solved? Information extraction design space for layout-rich documents using large language models*. arXiv. <https://arxiv.org/abs/2502.18179>

[16] Yang, Q., Hu, Y., Cao, R., Li, H., & Luo, P. (2021). *Zero-shot key information extraction from mixed-style tables: Pre-training on Wikipedia*. Proceedings of the AAAI Conference on Artificial Intelligence, 35(2), 1451–1456. <https://doi.org/10.1609/aaai.v35i2.16233>

[17] Raja, S., Mondal, A., & Jawahar, C. V. (2022). *Visual understanding of complex table structures from document images*. Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, 2299–2308.

[18] Maurya, C. K., Gantayat, N., Dechu, S., & Horváth, T. (2020). *Online similarity learning with feedback for invoice line item matching*. arXiv. <https://arxiv.org/abs/2001.00288>

[19] Oral, B., & Eryigit, G. (2022). *Fusion of visual representations for multimodal information extraction from unstructured transactional documents*. International Journal on Document Analysis and Recognition, 25, 187–205. <https://doi.org/10.1007/s10032-022-00399-3>

[20] Shneiderman, B. (2020). *Human-centered artificial intelligence: Reliable, safe & trustworthy*. International Journal of Human–Computer Interaction, 36(6), 495–504. <https://doi.org/10.1080/10447318.2020.1741118>

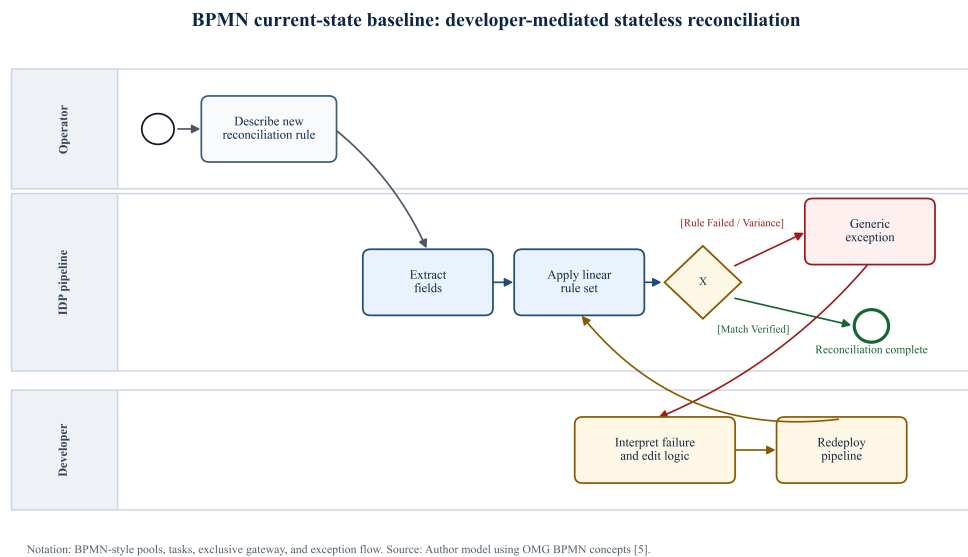
[21] n8n. (n.d.). *Nodes*. n8n Docs. <https://docs.n8n.io/workflows/components/nodes/>

[22] Langflow. (n.d.). *Langflow: A visual framework for building multi-agent and RAG applications*. GitHub. <https://github.com/langflow-ai/langflow>

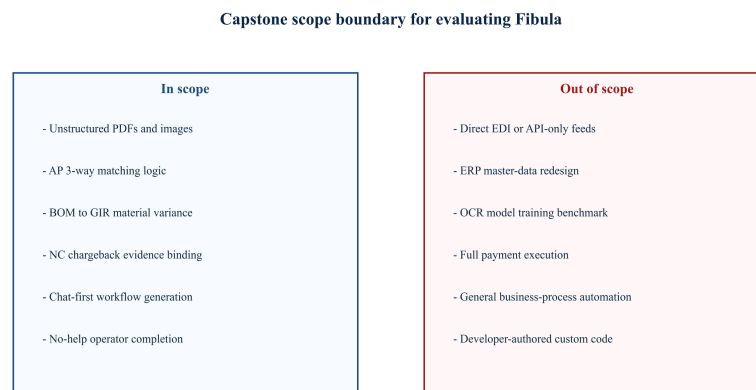
Appendices:

[1] Website: <https://jungluchen.github.io/Innovation-Xlab/>

[2] Figure 1. BPMN current-state baseline: developer-mediated stateless reconciliation. Source: Author model using OMG BPMN notation concepts [5].

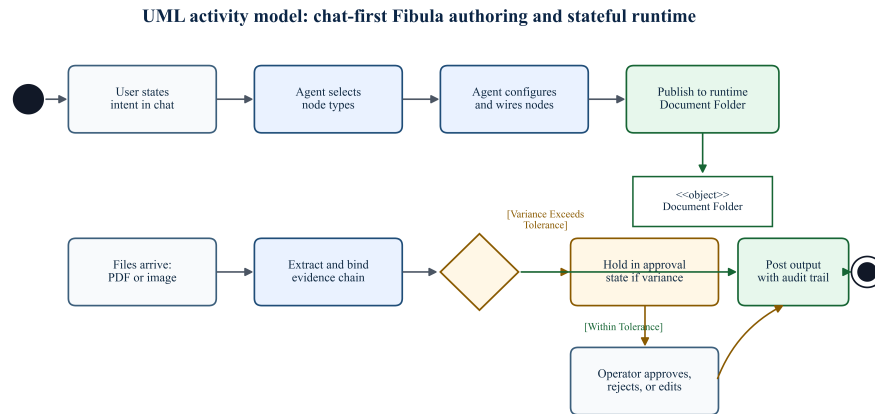


[3] Figure 2. Capstone scope boundary. Exclusions isolate the HCI variables of chat-first authoring, workflow state, and operator comprehension. Source: Author scope model based on capstone constraints and host-company document-processing context [1, 2].



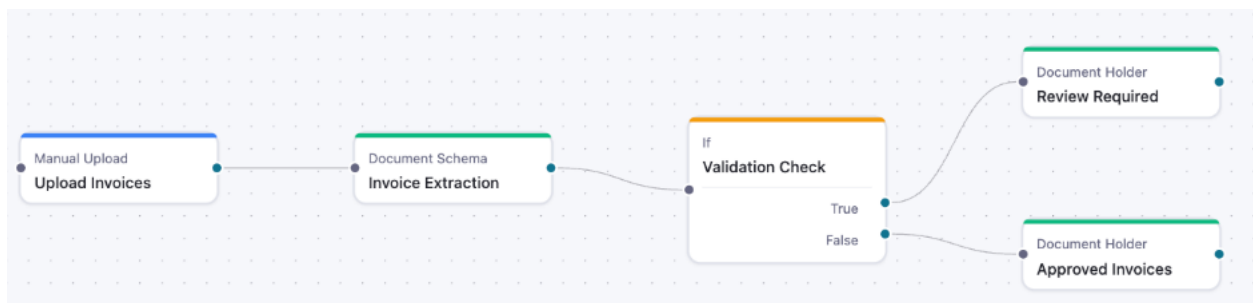
Source: Author scope model based on capstone constraints and host-company document-processing context [1, 2].

[4] Figure 3. UML activity model: chat-first Fibula authoring and stateful runtime. Source: Author model using OMG UML activity-diagram concepts [6].



Notation: UML activity nodes, object node, decision node, and final node. Source: Author model using OMG UML concepts [6].

[5] Figure 4 AP reconciliation process built in Fibula



Participant Consent Form

Fibula User Research Study - NUS Capstone Project with Staple AI

Project title	Extracting and Reconciling Industrial Operations Data with AI: Automotive Manufacturing and Supply Chain
Research team / sponsor	Innovation Xlab, NUS Capstone Project Team / Staple AI Pte Ltd
Purpose of study	To evaluate how users interact with Fibula, an AI-enabled workflow automation platform for document processing and reconciliation.
Expected duration	Approximately 30-60 minutes

1. Participation

You will be asked to complete realistic tasks using Fibula, such as starting a workflow, uploading or selecting documents, reviewing generated workflow nodes, testing a workflow, and locating or interpreting output results. You may also be asked short follow-up questions about your experience.

2. Voluntary participation

Participation is voluntary. You may refuse to answer any question, pause the session, or withdraw at any time without penalty. If you withdraw, you may request that your identifiable responses are not used in the report, where technically feasible.

3. Data collection, confidentiality, risks and benefits

- The research team may take written notes about your actions, comments, points of confusion, and suggestions.
- The study tests the usability of Fibula, not your personal ability or performance.
- Findings will be reported in aggregated or anonymized form. Your name will not be used in the final report unless you give explicit written permission.
- Please avoid entering real confidential business data, sensitive personal data, or information that you do not have permission to share.
- There are no expected physical risks. You may experience minor confusion or frustration while using the prototype; this helps identify where the application can be improved.

4. Consent statements

Tick	Consent statement
☐	I have read and understood the purpose of this study.
☐	I understand that my participation is voluntary and that I may stop at any time.
☐	I understand that I am testing the application, not being personally evaluated.
☐	I agree that the research team may take notes on my interaction, comments, and feedback.
☐	I agree that anonymized findings may be used in the final capstone report, presentation, and product recommendations.
☐	I understand that I should not upload or enter confidential business data, sensitive personal data, or information that I do not have permission to share.

5. Participant confirmation

Participant full name	
Participant signature	
Date	
Researcher name/signature	

[7] Interview snapshot samples

Quick Facts	Insights	Opportunities
Navigation Clarity: The "go-to workflow" button is easily observed and understood by users during the initial setup. Historical Access: Users struggle to locate previous chat sessions; they frequently miss the "New Chat" button and must be guided to find historical sections. Sample Data Issues: Sample files provided within the UI are difficult to read, lack zoom functionality, and often fail to download, preventing users from reusing them for testing. Silent Failures: The system occasionally fails during document uploads (e.g., Excel files) without providing clear reasons or actionable hints to the user.	Navigation Loops: The system redirects users to a general workflow tab rather than their active chat room, disrupting the logical flow of the conversation. Mental Model Mismatch: Users expect a persistent sidebar for historical chats, similar to ChatGPT, rather than needing to navigate through a "Builder" tab to find them. Testing Friction: The inability to download and reuse sample data significantly slows down the user's ability to run end-to-end tests smoothly. Technical Opacity: When automated agents attempt to "prepare" a document for download and fail, the lack of status updates leaves the user in an uncertain state.	Persistent Sidebar: Implement an expandable sidebar for historical chat sections, potentially including folders for organization, to align with industry-standard UX. Enhanced Document Viewer: Upgrade the sample data preview with browser-standard PDF viewing features, such as zooming and a high-visibility download button. Contextual Error Reporting: Replace generic dismissal options with detailed hints or "Next Step" advice when an upload or process fails. Direct "Resume Chat" Logic: Ensure that navigating back from a workflow canvas returns the user to the specific ongoing chat session rather than the general application landing page.
Quick Facts	Insights	Opportunities
The user initially checked the 'New Workflow' page, then immediately returned to the home page and selected the chatbot (the chatbot's presence was mentioned in the opening message). the user's task was to create a PO and invoice reconciliation, yet he uploaded a BOM and PO and continued with the process (the chatbot detected this and offered some solutions). After the workflow was generated, the user was unable to locate the entry point immediately. 	Fear of unfamiliar modification processes: Users appear at a loss when faced with the switch node, worried that if he made modifications to the case and the corresponding prompts the system will crushed. Furthermore, users do not understand the meaning of the prompts below the case label or how they should be written. In-depth questions were asked. Bug? Under the Reconciliation node -> Recent Documents path, there is an issue where resources cannot be displayed. Although the system renders the uploaded PO.pdf correctly, the BOM.jpg cannot be displayed correctly on the front end. 	Font size: Users of laptops would like to have a dedicated 'font scaling' function. Currently, adjustments can only be made via browser zoom, which disrupts the overall layout structure. Layout: - Under the RECONCILIATION -> Recent Documents -> line_items path, the current interface lacks flexibility. When reconciling data, users can only rely on the mouse wheel to scroll through the content, resulting in a less-than-smooth user experience. May be need to allow users to manually scale the display area vertically as required. - On the Workflows -> Logs page, users have reported that the current information display is overly crowded. Furthermore, the prompts at the top and bottom are inconsistent.

[8] Process table

Stage	Purpose in evaluation	Example user action	Evidence recorded
1. Entry and onboarding	Observe how users identify where to start	Open app, sign in , choose chatbot/builder, start workflow	First-click path, hesitation, entry choice
2. Workflow intention	Observe how users express the task goal	Describe extraction/reconciliation goal, select template	Prompt clarity, task interpretation
3. Document input	Observe how users provide source files	User dummy variables, upload different documents	Upload path, file-type issues, confidence

4.Workflow configuration	Observe how users inspect or modify logi	Review nodes, schema, matching rule, validation step	Node understanding, field interpretation, manual processing
5.Workflow testing	Observe how users validate the workflow	Run test, use dummy data, rerun	Overall understanding of success/failure
6.Output interpretation	Observe how users locate and act on results	Find result, check approval/review status, interpret output	Output visibility, next-action clarity

[9] AP 3 way matching evaluation logic

Document	PO, GRN, Invoice
Primary linkage key	PO_number
Supporting key	Item Code, Quantity, UOM, Unit Price, Currency, Vendor Name, Document Date
Core validation	ordered vs received vs invoiced consistency
Output	approve, review, reject, or hold payment

[10] BOM matching evaluation logic

Document	Work Order, BOM, GIR, Scrap log
Primary linkage key	Work Order ID
Supporting key	Item Code / Component Item / Part Number
Core validation	expected vs actual consumption
Output	variance identified, supplier chargeback, internal improvement report

[11]Table 2 Journey-level diagnostic metrics from unguided Fibula testing

Severity Index	Meaning	Testing Standard
P0	Critical blocker	The user cannot continue the task without help.
P1	Major usability failure	The user can continue, but selects the wrong path or creates a workflow that does not match the task goal.
P2	Moderate friction	The user can complete the task, but shows hesitation, repeated searching, or misses important information.
P3	Minor friction	The issue does not block the task, but reduces clarity, speed, or confidence.